

LLMs in Software Engineering: AI-Assisted Coding and Debugging

Presented by: V Rohan (192472227) | B.E – Computer Science and Engineering (AI)

SIMATS Engineering, Saveetha School of Engineering | Dept. of Computer Science and Engineering
Course: CSA65 - Generative AI and Large Scale Models | CO5: Assessment Tool 2 - Seminar / Poster Presentation

1. INTRODUCTION

Introduction & Background

- Large Language Models (LLMs) such as GPT, Claude and Codex-family models are trained on massive corpora of source code and natural language.
- What began as simple line-by-line autocomplete (e.g. GitHub Copilot) has evolved into agentic coding assistants that read, plan, write, test and ship code with minimal supervision.
- Tools like Claude Code, Cursor, Devin and OpenAI Codex now operate as autonomous software engineering partners across the full development lifecycle.

Problem Statement

- Manual coding, code review and debugging remain slow, repetitive and error-prone, especially in large, fast-growing codebases with complex, cross-service bugs.
- As developers increasingly turn to AI first for help, teams risk losing the organic knowledge-sharing that comes from asking colleagues.
- Real-world evidence on whether AI assistance actually speeds up delivery is mixed, so understanding where and how it genuinely helps is essential.

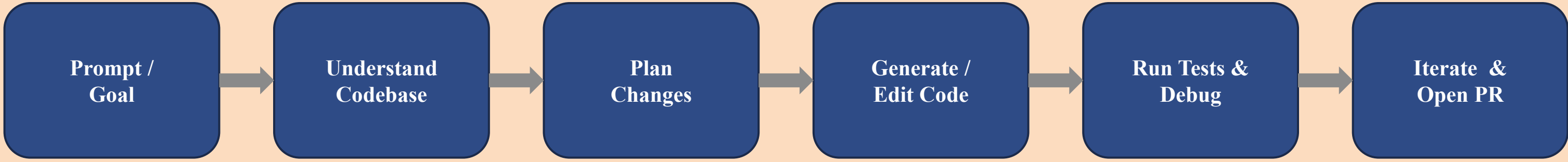
84% of developers now use or plan to use AI coding tools, and AI-generated suggestions make up an estimated 46% of code in instrumented environments (Stack Overflow / GitHub, 2025).

2. OVERVIEW & WORKING PRINCIPLE

Overview of the LLM Technology

Transformer-based LLMs fine-tuned on multi-language source code power two generations of tools: (1) inline assistants that suggest or complete code, and (2) agentic systems (e.g. Claude Code, Cursor, Devin, Codex) that autonomously understand a repository, plan multi-file changes, execute shell commands and tests, and open pull requests with limited human supervision.

Agentic Coding Workflow



The agent loops through the plan-edit-test steps, verifying its own work, until the task is complete.

3. APPLICATIONS

Industry Applications

- IT & Software Services - automated code generation, review and legacy modernization
- FinTech & Banking - large-scale codebase migrations and compliance-safe refactoring
- Enterprise SaaS - AI code assistants embedded in internal developer platforms
- E-commerce & Cloud Platforms - CI/CD-integrated agents for faster release cycles
- Startups & Product Teams - rapid prototyping and multi-file feature development
- Big Tech (Microsoft, Google) - organization-wide rollout of agentic coding tools

Real-World Use Case

Nubank x Devin (Cognition AI)

Digital bank Nubank used the autonomous coding agent Devin to migrate more than 100,000 data-class implementations.

A migration originally projected to take around 18 months was completed within a few weeks, illustrating the scale of efficiency an agentic coding system can unlock for large legacy codebases.

Enterprise trend: Microsoft's 2026 internal rollout of terminal-based agents such as Claude Code and GitHub Copilot CLI reflects a wider industry shift toward agent-assisted engineering at scale.

4. ADVANTAGES & CHALLENGES

Advantages

- Faster code generation and task completion - lab and vendor studies report 20-56% speedups on common tasks.
- Automates repetitive work: boilerplate, refactoring, documentation and test generation.
- Assists debugging by explaining errors and tracing root causes across multiple files.
- Agentic tools handle multi-step, multi-file tasks with minimal supervision.
- Lowers the barrier to onboarding and supports knowledge transfer for new developers.

Challenges / Limitations

- Evidence is mixed: a rigorous 2025 randomized trial found experienced developers were about 19% slower with AI despite expecting to be faster.
- Over-reliance on AI-generated code without review can introduce security vulnerabilities and compliance risk.
- Heavier use of AI-first help can erode team knowledge-sharing habits.
- Rising operational cost: LLM API usage typically dominates the running cost of enterprise coding platforms.
- Occasional hallucinated code and wasted agent runs still require careful human oversight.

5. FUTURE SCOPE

- Multi-agent, per-task model routing: strong models plan, cheaper models execute in parallel.
- Deeper integration into CI/CD pipelines and issue trackers (GitHub, Linear, Slack).
- Longer-running autonomous agents and stronger governance / evaluation frameworks.

References

1. Mohamed, A., Assi, M., & Guizani, M. (2025). The Impact of LLM-Assistants on Software Developer Productivity: A Systematic Review and Mapping Study. ACM Transactions on Software Engineering.

2. Kumar, A. et al. (2025). Intuition to Evidence: Measuring AI's True Impact on Developer Productivity. arXiv:2509.19708.

3. Weisz, J. D. et al. (2025). Examining the Use and Impact of an AI Code Assistant on Developer Productivity and Experience in the Enterprise. CHI EA '25, ACM.

4. Becker, J. et al. (2025). METR Randomized Controlled Trial on AI Coding Assistants, as reported in: The Productivity-Reliability Paradox: Specification-Driven Governance for AI-Augmented Software Development. arXiv:2605.01160 (2026).

5. Tembo (2026). Best AI Agents for Coding in 2026: A Developer's Guide. tembo.io.

6. Vellum AI (2026). 10 Best AI Coding Agents in 2026: Reviewed & Compared. vellum.ai.

Conclusion

LLM-assisted coding has moved from simple autocomplete to autonomous, agentic software engineering. It offers real productivity potential, but organizations must pair these tools with process changes, review discipline and governance to convert faster code drafts into genuinely faster, safer delivery.